

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Interdisciplinary Project in Informatics

**Graph Neural Networks for Hardware
Reverse Engineering**

Fabian Braun and Pauline Laßmann

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Interdisciplinary Project in Informatics

**Graph Neural Networks for Hardware
Reverse Engineering**

**Graphneuronale Netze für Hardware
Reverse Engineering**

Authors:	Fabian Braun and Pauline Laßmann
Supervisor:	Prof. Dr. -Ing. Georg Sigl
Advisor:	Johanna Baehr
Submission Date:	20.08.2024

I confirm that this interdisciplinary project is my own work and I have documented all sources and material used.

Munich, 20.08.2024

Fabian Braun and Pauline Laßmann

Abstract

This IDP implements reverse engineering for circuits using graph neural networks (GNNs) based on the concept of GNN-RE by Alrahis et al. (2022) [Alr+22]. This approach utilizes GNNs to analyze gate-level netlists and classify circuits based on their functionality. A GNN works by aggregating features and the graph structure of each node neighborhood. This allows the model to capture both local interactions and global structural information, enabling an effective approach to graph classification. Similar to GNN-RE [Alr+22] in this implementation, each node represents a gate and encapsulates the functional and structural characteristics of the surrounding gates in its feature vector. This IDP aims to test the concept of GNN-based circuit classification for a very large dataset. Initially, we validated our implementation using the dataset of GNN-RE [Alr+22] before testing with a much larger data collection provided by Fraunhofer AISEC. Our tests show mixed results with accuracies of up to 80% and are a notable decline in performance compared to GNN-RE. This highlights the importance of high-quality data while revealing challenges associated with scaling to larger, more complex datasets.

Contents

Abstract	iii
1 Introduction	1
2 Related Work	3
2.1 Current works	3
2.2 GNN-RE: Graph Neural Networks for Gate-Level Netlist Reverse Engineering	4
3 Our implementation	9
3.1 Datasets	9
3.1.1 GNN-RE dataset:	9
3.1.2 AISEC dataset:	10
3.2 Preprocessing	11
3.2.1 Feature extraction	12
3.2.2 Data splitting	12
3.3 GNN architecture and training	13
3.3.1 GNN architecture	13
3.3.2 Training	14
3.4 Postprocessing	15
4 Results	17
4.1 Example test result of the reduced AISEC dataset:	18
4.2 GNN-RE dataset tests	19
4.3 Configuration testing	19
4.4 Full dataset tests	21
4.5 reduced dataset tests	21
4.6 arithmetic and core components dataset tests	22
4.7 osu035 dataset tests	22
5 Discussion	24
6 Conclusion	26

Contents

List of Figures	27
List of Tables	28
Bibliography	29

1 Introduction

Over the last couple of years, continuous progress has occurred in the complexity of integrated circuit (IC) fabrication. The semiconductor manufacturing process is now globalized, with many companies worldwide supplying the various components required. This globalization, however, inevitably leads to several security vulnerabilities, such as malicious circuit insertion or modification, as well as intellectual property piracy. Because of that, the semiconductor industry heavily relies on providing hardware assurance and security. IC verification for malicious modification detection is just one example of such an assurance. Hardware reverse engineering represents a highly efficient method of analyzing and understanding the design and functionality of electronic circuits. It is typically employed at the logic gate level, although it is a costly process [RSK14], [Hon+21], [McL11].

The exponential growth in the complexity of circuit designs has turned manual hardware verification into an increasingly onerous task. Computer-aided hardware analysis aims to convert manufactured circuits into processable data so that they can be compared with the original design or analyzed at a functional level. Specifically for this analysis, an IC is unpacked and de-layered, after which high-resolution images of the stripped IC are captured. The extracted circuit connections enable the generation of a netlist for the circuit [Bot+21]. This netlist can then be analyzed to verify the design on a functional and structural level and, e.g., check for security vulnerabilities [Alr+22]. To analyze conventional heuristic methods, library-based matching methods with a cell library or knowledge-based embedding methods require human experience or are difficult to generalize to complex designs. Another approach to tackle these often large amounts of data is machine learning [Hon+21].

This is precisely the objective of this interdisciplinary project in informatics (IDP). This report will explain a Graph Neural Network (GNN) based reverse engineering model in Python in detail. The GNN-RE (Graph Neural Networks for Reverse Engineering of Gate-Level Netlists) [Alr+22] serves as a template. The project aims to implement the model used in [Alr+22] to verify the results using their dataset, test its performance, and scale on another large dataset using various benchmarks.

The remainder of the report is structured as follows: First, related works, as well as the paper this project is based on, are discussed in section 2. Then, our implementation is detailed in section 3. Lastly, this report presents and discusses our results in sections

4 and 5 before concluding in section 6. Additionally, this report will highlight the individual contributions of each team member to the project.

2 Related Work

Machine learning has yielded remarkable outcomes in numerous domains, particularly its capacity to process vast data. Recent work in this field has employed machine-learning techniques for circuit classification.[Hon+21]

Section 2.1 presents current works on machine-learning-based circuit classification. In section 2.2, the paper this IDP is based on, and its contributions and results are reviewed in detail.

While both team members independently reviewed relevant literature to gain insights into the topic and understand the current state of research in the field, each team member focused on learning specific, more advanced information about a topic corresponding to their implementation part. In detail, Pauline Laßmann focused on the more advanced attention mechanism used in graph attention networks (GAT), and Fabian Braun concentrated on the random walk sampling necessary for scaling to large data.

2.1 Current works

Silva et al. (2018) employed a convolutional neural network (CNN) to classify arithmetic circuits by analyzing integrated circuit images from a circuit’s conjunctive normal form description [Sil+18]. The authors manage a high accuracy of over 80% in mapping a macro functional block as they propose an innovative identification of functional blocks through images. The system is implemented based on CNN using TensorFlow.

In 2019, Fayyazi et al. presented a scalable framework for gate-level circuit recognition based on deep learning and a convolutional neural network (CNN) [Fay+19]. A sparse mapping algorithm is used to improve the time and memory efficiency of the CNN-based circuit representation. Furthermore, a termed level-dependent decaying sum (LDDS) existence vector is introduced, which compactly represents information about the circuit topology. The demonstrated framework reduces the average training time and improves the average run time for generating CNN-based representations from gate-level circuits.

In their work, Dai and Brayton (2017) show a new circuit representation for the circuit-based convolution operation with dynamic pooling [DB17]. Experiments are done for circuit classification, function detection, and localization with a deep learning

framework utilizing CNNs to recognize circuit functionalities, demonstrating the effectiveness of the proposed method compared to reference methods based on support vector machines (SVM). A model can be trained with this proposed framework to help detect and locate malware in hardware designs with appropriate training data, such as a set of circuits with hidden trojans.

Both the work of Kunal et al. (2020) [Kun+20] and Li et al. (2020) [Li+20] employ a graph neural network for the classification of analog circuits, utilizing a graph pooling strategy. The approach in Kunal et al. (2020) [Kun+20] uses a trained graph convolutional neural network (GCN). It identifies netlist elements for circuit blocks at upper levels of the design hierarchy, while structures at lower levels are identified using graph-based algorithms. A recognizable accuracy can be obtained for sub-circuit identification over a wide range of analog designs. Larger circuits can also be successfully identified, and hierarchical decompositions of such circuits can be provided. In the work of Li et al. (2020) [Li+20], their customized graph neural network model predicts the impact of placement on circuit performance. Their results of the simulations show that the proposed model is superior to recent CNN-based work in terms of accuracy and knowledge transfer. Furthermore, performance similar to manual designs is archived with the proposed model when applied in analog IC placement.

Hong et al. (2021) [Hon+21] presents a graph neural network-based machine learning model for automated circuit classification. Performed experiments showed that the machine learning model has up to 98.3% accuracy for predicting accurately. As an example, a study on adder circuits was conducted, showing that GNN can successfully differentiate adder circuit graphs within two hops, demonstrating that the model can perform logical differentiation of circuits by analyzing their structure.

2.2 GNN-RE: Graph Neural Networks for Gate-Level Netlist Reverse Engineering

The paper "GNN-RE: Graph Neural Networks for Gate-Level Netlist Reverse Engineering" by Alrahis et al. (2022) [Alr+22] lays the basis for this IDP. It introduces GNN-RE, a machine learning-based model that employs graph neural networks for the functional reverse engineering of circuits. For this, flattened gate-level netlists are analyzed. The graph-based representation of circuits allows for consideration of both the global structure and local neighborhoods. This way, sub-circuits can be identified and labeled. With the used datasets, GNN-RE attains an average accuracy of 98.82% in mapping gates to modules without the need for manual intervention.

The contributions and results of the paper are presented now.

A gate-level netlist is represented as an undirected graph $G = (V, E)$, where V is the set

of nodes (gates) and E is the set of edges (wires) connecting the nodes. Additionally, each node in the graph has a vector of length k as an attribute representing the node features. The feature vector includes connectivity to primary in- and outputs, the node's functionality, and those of its neighboring nodes for up to two hops. Additionally, the features contain information about the node's number of in and outgoing connections, making it possible to view the netlist as an undirected graph without losing structural information. Since the feature vector includes information about connected gates, its size, k , depends heavily on the number of gate types in the used technology library. The classification done by GNN-RE is achieved by learning from the graph structure and aggregating features, capturing the functionality of a neighborhood. This allows GNN-RE to absorb any given node's structural and functional attributes and surrounding circuitry. Ultimately, GNN-RE can perform node classification and generate a prediction for each sub-graph selected from an original graph.

According to optimization testing by the authors, the optimal GNN architecture for GNN-RE uses a graph attention network (GAT) with four convolutional layers for embedding computation and a fully connected layer for classification. Furthermore, their optimal GNN uses a hidden dimension of 256, a dropout rate of 0.1, and ReLU activation. For the training of their GNN, the authors use the GraphSaint [Zen+19] random walk sampling approach. The concept of random walk sampling involves traversing a graph by moving from one node to a randomly chosen neighboring node at each step. This process continues from a root node for a predetermined number of steps or until a stopping criterion is met. Random walk sampling plays a key role in efficiently training with large-scale graphs since the whole graph does not have to be used while still maintaining a representative overview. To utilize this concept in a dataset of multiple graphs, all graphs are first combined into a single structure. In GNN-RE, the process first involves using GraphSaint's random-walk sampler with a walk length of 2, sample coverage of 50, and 3,000 root nodes to construct sub-graphs from the training set that will be used for training. During sampling, the random walk probabilities are calculated for normalization as proposed in [Zen+19] to ensure nonbias against nodes less frequently sampled. The model is then trained for 2,000 epochs using the Adam optimization algorithm with a learning rate of 0.01.

The GNN-RE dataset used for sampling includes various designs, such as sub-circuits with varying interconnections and bit-width sizes, which are listed in the next paragraph about the datasets used. An overview of GNN and the sampling parameters can be seen in table 2.1.

The dataset used is comprised of combinational and sequential circuits. Regardless of whether boundary identification and sub-circuit extraction are required, GNN-RE can work under both conditions, with the former being favored. In the latter case, single modules are used that contain designs that represent either an adder, subtractor,

Architecture		Training and Sampling	
Total # Layers	5	Optimizer	Adam
GNN Depth	1	Dropout	0.1
Input Layer	[33, 256]	Attention	8
Hidden Layers 1-4	[512, 256]	Learning Rate	0.01
Output Layer	[256, #classes]	Sampler	Random Walk
Aggregation	Attention aggregator with multi-head concatenation	Walk Length	2
Activation	<i>ReLU</i>	Root Nodes	3000
Classification	<i>softmax</i>	Max # Epochs	2000

Table 2.1: GNN-RE configuration details, table taken from [Alr+22]

multiplier, or comparator, as well as sequential modules that include counters, shift registers, and finite state machines. The datasets where boundary identification and sub-circuit extraction are required are combined in the interconnected-modules dataset and consist of:

- An add-mul-mux dataset: designs consisting of one adder, one multiplier, and control logic to perform arithmetic operations based on inputs
- An add-mul-mix dataset: designs with two adders whose outputs feed into a multiplier
- An add-mul-combine dataset: designs with one adder and one multiplier sharing the same primary inputs but affecting different primary outputs
- An add-mul-all dataset: combines designs from the previous three
- An add-mul-comp dataset: designs with a single adder, a single multiplier, and a comparator
- An add-mul-sub dataset: designs containing a single adder, a single multiplier, two subtractors, and control logic
- An add-mul-comp-sub dataset: designs where a comparator block dictates operations to be carried out by the main module

The used datasets are summarized in figure 2.1.

The performance of GNN-RE is measured by comparing the node predictions to the ground-truth labels associated with the corresponding databases. Four metrics are

2 Related Work

Case Study	Datasets	#Classes	#Features	#Nodes	#Circuits	#Training Nodes	#Validation Nodes	#Testing Nodes
I	Single-Modules	4	33	32,528	27	6,318	357	25,853
	Sequential-Modules	3	39	841	29	700	69	72
	Add-Mul-Mux	3		15,582	7	14,467	239	876
	Add-Mul-Mix	2		21,602	6	19,377	576	1,649
II	Add-Mul-Combine	2		14,288	6	13,256	217	815
	Add-Mul-All	3		51,472	19	47,100	1,032	3,340
	Add-Mul-Comp	4	33	15,898	6	14,717	255	926
	Add-Mul-Sub	4		18,206	6	16,786	309	1,111
	Add-Mul-Comp-Sub	5		19,151	6	17,648	342	1,161
	Interconnected-Modules	5		104,727	37	96,251	1,938	6,538

Figure 2.1: Dataset overview used by the authors, figure taken from [Alr+22]

calculated: accuracy, non-averaged precision, non-averaged recall, and non-averaged F1-Score. The impact of different GNN structures and hyper-parameters on accuracy and training time is performed and concludes in the above-mentioned parameters as they perform best. GraphSAGE is evaluated where the above-mentioned 256 hidden dimensions produced a Macro-F1 of 95.99% and a Micro-F1 of 90.87%. For a GNN depth of 2, a Micro-F1 score of 96.65% and a Macro-F1 score of 91.84% are achieved. However, when setting the depth to 4, the macro-F1 ratio degrades to 90.48%, leading to a depth choice of 1. The layers set to 4 achieve a Micro-F1 score of 97.54% and a Macro-F1 score of 91.83%. Other architectures like GAT, GAAN, and JKnet are examined as well, finding GAT to perform best, reaching a Micro-F1 score of 97.84% and a Macro-F1 score of 95.99%.

The single-modules dataset at the node level achieves a classification accuracy of 99.28% with precision and recall values ranging from 87.29% to 100% for a node. After post-processing, it adapts that to full graph classification that checks the percentage of predicted node classes. It then labels the graph based on the most predicted class, achieving a sub-circuit classification accuracy of 100% for all the tested designs.

Case Study	Dataset	GNN Accuracy (%)	Precision (%)				Recall (%)				F1-Score (%)				Training Time (s)			
			ADD	MUL	CTRL	COMP	SUB	ADD	MUL	CTRL	COMP	SUB	ADD	MUL		CTRL	COMP	SUB
I	Single-Modules	99.28	95.15	99.99	-	98.35	87.29	100	99.34	-	98.73	97.75	97.52	99.67	-	92.66	98.05	1,282.02
	Add-Mul-Mux	99.32	97.43	99.60	97.50	-	-	96.20	99.60	100	-	-	96.82	99.60	98.77	-	-	2,289.13
	Add-Mul-Mix	99.45	99.37	99.47	-	-	-	98.43	99.85	-	-	-	98.9	99.66	-	-	-	3,119.42
	Add-Mul-Combine	99.02	90.12	100	-	-	-	100	98.92	-	-	-	94.87	99.45	-	-	-	2,297.50
II	Add-Mul-All	97.75	88.61	99.5	95	-	-	97.25	98.23	88.37	-	-	92.73	98.72	92.68	-	-	6,838.18
	Add-Mul-Sub	98.02	86.42	99.61	90.91	-	100	93.33	99.74	98.77	-	92.55	89.74	99.67	94.67	-	96.13	2,296.61
	Add-Mul-Comp	99.35	94.59	99.87	97.06	100	-	100	100	97.06	89.13	-	97.22	99.94	97.06	94.25	-	2,275.00
	Add-Mul-Comp-Sub	98.28	100	97.42	100	100	100	100	100	91.14	92.9	100	100	98.69	95.36	96.32	100	2,308.70
	Interconnected-Modules	98.87	97.51	99.14	94.74	100	100	96.24	99.61	97.5	97.39	94.88	96.87	99.38	96.1	98.68	97.37	16,317.44

Figure 2.2: GNN-RE node classification result, figure taken from [Alr+22]

When testing GNN-RE on the datasets with interconnected sub-circuits, the highest performance was achieved for the add-mul-mux dataset with 99.45% and the lowest performance for the add-mul-all dataset with 97.75%. An increased number of considered classes did not hinder the model’s performance, but an improvement in the

F1-Score by 4.14% for the ADD classifier and by 0.66% for the MUL classifier could be observed. The interconnected-modules dataset with a total number of 104,727 nodes compared to the add-mul-comp-sub dataset with 19,515 nodes increased the accuracy from 98.28% to 98.87%. Figure 2.2 shows an overview of the results.

Lastly, mispredicting multiplication nodes as adder nodes was the most frequent cause of misclassification observed in 40% of all cases. This is followed by mispredicting adder nodes as multiplication nodes with 18% of all cases. Overall, node misclassification remains under 2.25% for any dataset.

3 Our implementation

In this section, we present our Python implementation of hardware reverse engineering using a graph neural network, which is heavily based on the work by Alrahis et al. (2022) [Alr+22]. This aims to reproduce and validate the results described in section 2.2 and test the solution’s performance and scaling on another much larger dataset.

First, the datasets used for this project are presented in section 3.1. Next, our preprocessing is described in section 3.2. Then, our main GNN and its variants are discussed in section 3.3 before lastly, the postprocessing of the GNN outputs is presented in section 3.4.

3.1 Datasets

This section gives an overview of the datasets used during our project. We differentiate two main datasets. First, we use the combined "Interconnected-Modules" dataset used in [Alr+22]. Second, we use a large set of circuit designs synthesized with different cell libraries. This second dataset was provided by Fraunhofer AISEC. Throughout our project, we selectively filtered the AISEC data to focus on significant data and created some subsets based on subjective criteria to test various cases.

The initial busywork of filtering and fitting the large AISEC dataset to our needs was shared among the whole team by iteratively solving needed tasks like removing unnecessary graphs causing specific imbalances in the data. The selection of the specific subsets was carried out collaboratively by the team to reach a consensus on the experiments that could be conducted and should be prioritized.

3.1.1 GNN-RE dataset:

This dataset is the publicly available "Interconnected-Modules" dataset¹ that is used by the authors of [Alr+22]. See 2.2 and 2.1 for a detailed overview of its original contents. However, since we used a different parser to process the dataset’s source verilog files combined with our own implementation for feature extraction, the resulting final data differs slightly from the original by using a different feature vector as seen in table 3.1.

¹<https://github.com/DfX-NYUAD/GNN-RE>

Dataset Version	#Classes	#Features	#Nodes	#Circuits
as used in GNN-RE [Alr+22]	5	33	104,727	37
using our implementation	5	20	104,727	37

Table 3.1: GNN-RE dataset [Alr+22] differences from feature implementation

3.1.2 AISEC dataset:

The Fraunhofer Institute AISEC provided us with a large collection of circuit-netlists. These netlists were synthesized using one of four cell libraries: osu035, osu018, nangate, and gscl45nm. For many of the designs, the data contains multiple versions across the different cell libraries, giving the opportunity to test and evaluate GNN performance across the different technologies.

The biggest challenge of this dataset was that the data was not labeled. After some ineffective attempts to extract meaningful labels from filenames, the labeling was then done by using a catalog also provided by AISEC, mapping numerous designs to rough categories, including a large portion of our dataset. We further condensed the categories into more generalized labels by combining categories that are functionally similar or belong to the same hardware components. We filtered out labels with too little data available and a number of graphs that were just very slight variations of other graphs and were causing an imbalance in the dataset. This process left us with 27 labels across 1,768 netlist-graphs, as seen in table 3.2. These are graph-level labels, meaning a label describes a netlist as a whole. However, our project aimed to utilize the concepts of the [Alr+22], which performs node classification. Therefore, we extended the general label of a netlist-graph to each of its nodes.

The graph nodes contain gate information, which we use to extract a vector of 16 features for each node. Feature extraction is detailed in section 3.2

Additionally, we defined some subsets of this dataset to test and evaluate different data configurations. These subsets are:

reduced set: This subset contains fewer labels by only using graphs with the most interesting labels for us to identify. It includes, among others, labels like "memory" and "crypto" and labels for arithmetic circuits. We created this subset to have a slightly smaller set with more focused data that enables more efficient testing even in the case of a high number of test runs for extended experiments. The subset outline can be seen in table 3.2.

osu035 set: This subset only contains graphs that are synthesized using the osu035 cell library. Additionally, it also only uses the same labels as the reduced set. We use this

subset to test training on data from only one technology and its effects on classifying graphs from a different cell library. The subset outline can be seen in table 3.2.

arithmetic circuits set: This small subset contains only labels for arithmetic circuits like "add" and "mult". This subset aims to test if a small and highly curated set of labels significantly impacts GNN performance. The subset outline can be seen in table 3.2.

core circuits set: This small subset contains labels for circuits that have often used functionality and are not a better fit for another subset, like arithmetic functionality in the arithmetic subset. Example labels in this set are "memory" or "control". The goal here is to provide another set for testing on a small set of specific labels, just as with the arithmetic set. This way, we try to get an insight into not only how the number of labels has a performance impact but also how the similarity of labels could impact the results. The subset outline can be seen in table 3.2.

Dataset	#Classes	#Features	#Nodes	#Circuits
full dataset	27	16	7,503,589	1,768
reduced set	13	16	5,548,470	1,247
osu035 set	13	16	3,328,129	628
arithmetic circuits set	8	16	2,814,577	653
core circuits set	5	16	2,733,893	594

Table 3.2: AISEC dataset and subsets

3.2 Preprocessing

This part will give an overview of our preprocessing implementation. Our preprocessing includes functionality to read graphs and compute a feature vector for each node using the concept described in the work of Alrahis et al. (2022) [Alr+22] as well as dynamically splitting the dataset into training-, validation-, and test-set accordingly. Additionally, some functionality to store and load preprocessed graphs with their features and labels in a binary format is implemented here. This dramatically reduces run time after the dataset is processed once.

Fabian Braun implemented preprocessing and focused on enabling efficient testing even for vast data in addition to providing the core functionality described in this section.

3.2.1 Feature extraction

The feature uses the same structure as the vector in GNN-RE [Alr+22]. This means it contains information about the gate itself and its connected neighborhood. We use a default feature depth of two, meaning the neighborhood gathered in the node features includes gate information of up to two hops from the node. The vector is comprised of three main parts. First is the amount of primary inputs and outputs connected to the node within the feature neighborhood. Second, the number of gates for each possible gate type connected to the node within its neighborhood. This also includes the gate type of the node itself. Our first tests used a vector consisting of 54 different entries. This large feature vector was because multiple specific variants of some gates were tracked in the vector. However, this did not seem to result in a performance benefit. Now, so as not to unnecessarily inflate the size of our feature vector, we only track generalized gate types. Therefore, for example, an AND3 gate and an AND4 gate are counted as AND gates. The last part of the feature vector contains the amount of direct incoming and outgoing connections for the node. This information preserves the structural information from the netlist and enables us to use an undirected graph. Figure 3.1 shows the entire feature vector we use for the AISEC dataset.

After the graph features are computed during preprocessing, all nodes corresponding to a primary input or output are removed from the graph since they should not be considered for node classification by the GNN. The last preprocessing step is normalizing the features and adding the overall graph label to each node.

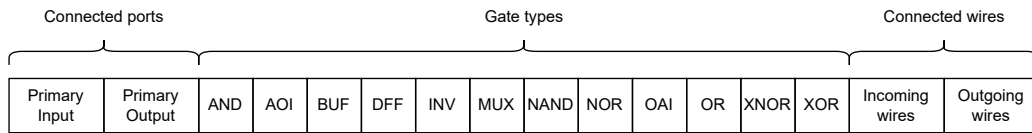


Figure 3.1: Full feature vector for AISEC data

3.2.2 Data splitting

The extensive AISEC data is not split into dedicated training, validation, and testing roles. Instead, before training, our implementation assigns these roles dynamically by randomly choosing graphs for each role. This is done for each label individually by taking 75% of the available graphs for training, 10% for validation, and the remaining 15% for testing. Dynamically assigning this way enables the option to easily use different subsets within our large dataset without the need for individually setting roles for each of the subsets. It also ensures that graphs are only assigned one role,

and consequently, a graph for testing is not known to the GNN from training, which provides realistic results. The drawback of this implementation is that some imbalances in our data can significantly impact some test runs since role assignment is entirely random. We try to counteract these unavoidable imbalances in the large dataset by calculating class weights from the label distribution in our training data. The loss function can then use these class weights during training. Additionally, we later averaged results over multiple test runs during evaluation. The preprocessing step of dynamically choosing data role also includes combining the training data into a singular graph structure, which is necessary to do random walk sampling of the whole dataset for training.

3.3 GNN architecture and training

In this section, our GNN architecture and training configurations are discussed. With the information given in the paper about GNN-RE [Alr+22], we tried to implement our GNN as close as possible.

However, we only implement a GraphSAGE [HYL17] and graph attention network (GAT) [Vel+17] model and don't implement tests with GAAN and JKnet as this exceeds the scope of this IDP. Moreover, while we try to build our model as close as possible to the GNN-RE model, there are some simplifications and differences. One of these is that we do not use a separate classifier for the GNN. Instead, our GNN directly maps its internal embeddings to the output classes on its last layer. Another difference is in the training algorithm, where we do not pre-sample the training subgraphs to calculate correct random walk probabilities. Subsequently, we cannot calculate the same normalized loss as used in GNN-RE [Alr+22].

Implementing the GNN model was cleanly split between the different models. This meant that the more basic graphSAGE model, together with the random walk sampling for more efficient testing, was implemented by Fabian Braun. In contrast, Pauline Laßmann implemented the more complicated GAT model. Initially, this meant that each model had an individual training function and execution framework. Later in the project, we were able to generalize this implementation with some teamwork into a general training function used by both models and a single execution framework that can start tests on both models and is configured by a configuration file. This also enabled using the more scalable random walk sampler for the GAT model.

3.3.1 GNN architecture

We use a default architecture for our GNNs, as shown in table 3.3. However, for some experiments, we adapt the architecture with the parameters listed in parenthesis in the

table. All major parameters for the GNN architecture can be easily configured in a config file before each test with our implementation. We use this flexible configuration to enable different test cases and more efficient testing by fitting the parameters to our available computing resources, especially in cases where a slight difference in a parameter does not seem to significantly impact GNN performance.

As mentioned above, our implementation includes GNNs with GraphSAGE or GAT as possible convolutional layers. The default GNN convolutional layer we use is GraphSAGE, which is less resource-heavy.

Furthermore, we aimed to use GNN parameter values as close as possible to the parameters used in GNN-RE [Alr+22] and focused our testing on similar values. However, we found that not all values achieved as good a result as in the paper. Therefore, we decided to modify some parameter values to achieve better results with our data. We are using the hidden dimension of 256, which was deemed optimal for GNN-RE. However, especially for GAT, which has an attention mechanism, many tests use a hidden dimension of 128 due to hardware constraints. Initial tests did not show a significant impact on results due to this change. The parameter values from GNN-RE that we could successfully adopt are the learning rate of 0.01 and the dropout rate of 0.1.

The number of layers was adjusted from 5 to 3 total layers, including the fully connected layer.

3.3.2 Training

For training, we again try to use a similar configuration to GNN-RE [Alr+22]. This primarily means that we also use the Random-walk sampler from GraphSAINT [Zen+19] with a walk length of 2 and 3000 root nodes. Since the primary dataset for our implementation is significantly larger than what was used in GNN-RE, we also tested different sampling configurations, like more root nodes and longer walks, to provide better dataset coverage. However, there was no measurable difference in results, so we continued using the original GNN-RE configuration. We also use the Adam optimizer. Additionally, we use the CrossEntropy loss function with a weight decay set to $1e-5$. For most experiments with GAT, the number of epochs was reduced to 500 or 1000 from 2000 used in GNN-RE [Alr+22] since it allowed for faster testing and, with our data training beyond a certain number of rounds, did not provide additional benefit. For graphSAGE, however, the 2000 epochs were always used and seemed to still bring at least a small performance benefit.

Configuration values for training can again be seen in table 3.3

Architecture		Training and Sampling	
GNN model	GraphSAGE (GAT)	Optimizer	Adam
Total # Layers	3 (4, 5)	Loss function	CrossEntropy
Input Layer	[16, 256]	Attention	8
Hidden Layers	[256, 256] (128, 64)	Learning Rate	0.01
Dropout Rate	0.1	Weight decay	1e-5
Output Layer	[256, 27] (13, 8, 5)	Sampler	Random Walk
Aggregation	Attention aggregator with multi-head concatenation	Walk Length	2
Activation	<i>ReLU</i>	Root Nodes	3000
Classification	<i>softmax</i>	Max # Epochs	500 (1000, 2000)

Table 3.3: GNN architecture and training configuration details

3.4 Postprocessing

To improve on the results produced by our GNN, the labeled graph runs through a round of postprocessing. Here, the graph is analyzed again, focusing on possibly correcting incorrectly predicted labels of nodes by considering the remaining graph and its node labels. Two options are available for this postprocessing. A node’s label can either be checked for possible wrongful labeling by considering the majority of labels or the probability of labels.

Pauline Laßmann implemented the postprocessing, focusing on converging towards the majority label while avoiding error propagation. The reason for this is the fact that our model performs node classification while our data uses graph-wide labels.

Majority based post-processing: The majority-based postprocessing checks every node in a graph. It considers the majority label of neighbors (the label set for most neighboring nodes of a checked node) and the overall majority label of the labeled graph (the label used the most over the whole graph). Two cases are considered.

First, the label of the checked node is unequal to the majority label of neighbors. If this is the case, it is checked if the majority label of neighbors occurs more often than half the total number of neighbors. Additionally, suppose the majority label of neighbors is the same as the overall majority label of the graph. In that case, the label of the checked node is adjusted to the majority label of neighbors.

Second, the label of a node is equal to the majority label of neighbors. If this is the case, it is checked if the overall majority label of the graph is unequal to the majority label

of neighbors. Additionally, suppose the majority label of neighbors occurs equal to or less often than half the total number of neighbors. In that case, the label of the checked node is adjusted to the overall majority label of the graph.

When neither of both cases and all conditions are met, the checked node retains its assigned label.

Probability based post-processing: Probability-based postprocessing checks every node in a graph. It considers the probability of a node’s chosen label and its standard deviation. Furthermore, the overall standard deviation of the graph and the mean probability of the label with the highest mean probability for the whole graph are considered.

In detail, this probability-based postprocessing works by first considering the checked node’s standard deviation. It is considered low when it is lower than 0.1 times the node’s mean probability of all labels. If this is the case and the overall standard deviation of the whole graph is bigger than 0.1, the node’s label is considered for label changes. Secondly, it is now checked if the probability for the label of the checked node is smaller than the highest overall mean probability. If this is the case, the label of the checked node is changed to the label with the biggest overall mean probability. Otherwise, the designated label given to the node by the GNN is retained.

The checked node will retain its assigned label if these conditions remain unfulfilled.

4 Results

During our project, we conducted a series of tests using the various sets outlined in table 3.2 using the configurations overviewed in table 3.3. All tests were performed on the same system with eight cores, 32GB RAM, and one GPU with 10,240 CUDA cores and 12GB GDDR6X memory. We measured the test performance by comparing the true labels of our data with the GNN-predicted labels. Then we calculated the same four metrics: accuracy, f1-score, precision, and recall used in GNN-RE [Alr+22].

The results overall were very mixed across the different datasets and notably worse than the results of using our implementation with the GNN-RE dataset. Additionally, there was some considerable variability for most tests, meaning that results changed by sometimes up to 15% between different runs of the same test. Since further investigation of this variance was outside this project’s scope, we can only assume that the reason for it lies in randomly choosing the data for training, validation, and testing roles, as well as the randomness introduced by the random walk sampler during training. An overview of the best results for each dataset and each GNN model can be seen in table 4.1.

Dataset	GraphSAGE				GAT			
	Accuracy (%)	F1-Score (%)	Precision (%)	Recall (%)	Accuracy (%)	F1-Score (%)	Precision (%)	Recall (%)
full dataset	41.38	20.97	24.08	32.97	34.93	24.99	28.07	44.63
reduced set	80.29	58.58	56.88	71.25	72.76	48.94	46.41	59.40
osu035 set	35.70	29.59	37.35	37.54	27.04	21.72	26.41	34.63
arithmetic circuits set	59.96	53.74	50.45	68.24	77.13	62.65	57.97	78.26
core circuits set	81.60	62.82	57.31	78.08	95.38	80.64	77.43	85.37

Table 4.1: GNN performance results of AISEC dataset and subsets

Generally, we can see that the performance for both models is similar, especially considering the significant variance. Performance for more extensive, more complex tests is poor, but we see improved results, especially for subsets with fewer classes. While the simpler graphSAGE model seems to have a slight edge in more complicated tests, it is bested by GAT in the smaller test. This is most likely due to the imbalances and overlaps in our large dataset. Reducing the number of classes for the smaller sets should remove large parts of these problems.

The ensuing paragraphs will elaborate on the test results to provide a comprehensive account. First, a specific example is presented in section 4.1. Then, the test results for different testing scenarios with the GAT GNN and the GraphSage GNN are discussed in sections 4.2 to 4.7.

4.1 Example test result of the reduced AISEC dataset:

This paragraph shows the results of one example executed test on the reduced AISEC subset containing 13 classes and 1,247 graphs in detail. This test utilized the GAT model configured with three layers, a hidden dimension of 128, and trained over 500 epochs. The evolution of our calculated metrics during training can be seen in figure 4.1 in addition to training and validation loss.

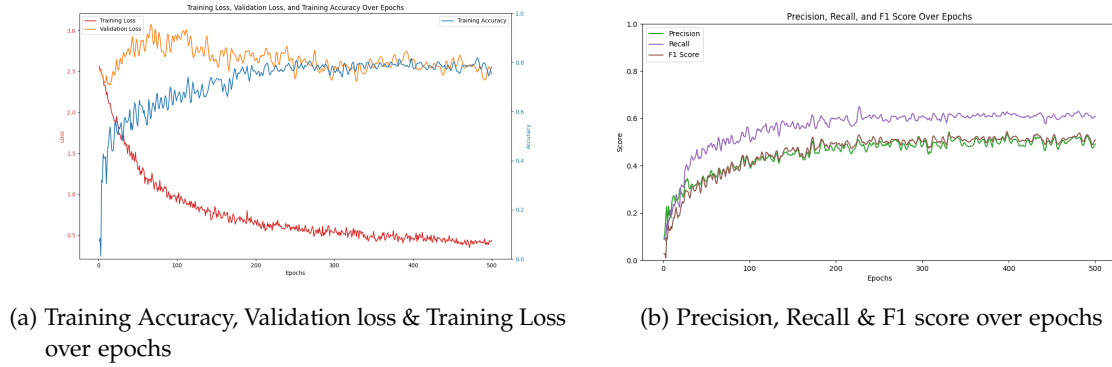


Figure 4.1: Metrics evolution during training of GAT on the reduced AISEC dataset

As illustrated in 4.1a, the validation accuracy quickly improves during training to around 80%. However, the graph also shows that its value is not very stable, and after testing, the test's overall accuracy comes to 72%. While the development of accuracy and training loss during training show generally expected behavior, we can also see that the validation loss does not decrease, indicating that the model does not generalize

well and hinting at imbalances and mismatches within our data. Figure 4.1b shows the precision, F1 score, and Recall during training that, while behaving normally, already plateau after around 250 epochs. The precision and F1 scores follow the same gradient and reach values that are also represented by the overall scores after testing, 46% and 48%, respectively. The recall score plateaus at a slightly higher value, which again reflects the testing score of 59%. While these metrics are generally lower than desired, the accuracy achieved is good.

Figure 4.2 illustrates exemplary predictions on one graph of a subtractor with the label "sub". Figure 4.2b shows the correctness of the model predictions before postprocessing. Here, we can see that while a good number of nodes were correctly predicted, there are also many wrongly predicted nodes. After postprocessing, the final predicted node labels are shown in figure 4.2a where silver corresponds to the label "sub". Consequently, postprocessing was effective in figure 4.2c, which again shows the correctly predicted nodes after postprocessing. While there is still a noticeable number of wrongly predicted nodes, it is evident that the majority is correctly predicted, meaning we can consider this graph correctly identified.

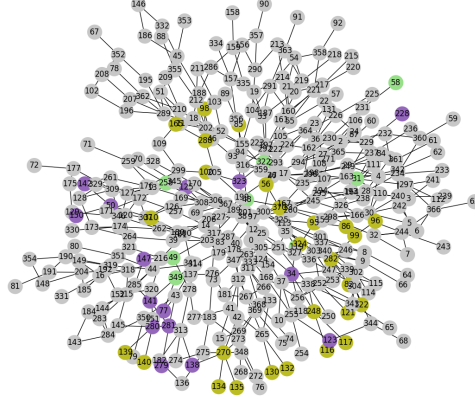
4.2 GNN-RE dataset tests

The first tests we conducted during our project used the "interconnected modules" dataset used in GNN-RE [Alr+22]. As shown in table 3.1, this dataset has 20 features and 37 circuits in our implementation. We used this dataset to validate our implementation by checking if it can achieve performance scores comparable to GNN-RE. Using our graphSAGE model, we achieved an accuracy of 97%, which is very close to the GNN-RE results. With 88% as the f1 score, 94% as precision, and a recall of 84%, we also reach good values in other metrics, even if these values do not entirely reach GNN-RE levels. The reason for this is most likely better optimization in GNN-RE. Nevertheless, we count our implementation as successful and able to replicate the GNN-RE results.

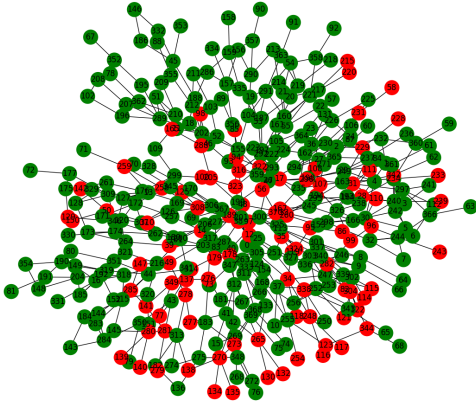
The same is true for our GAT model, which also achieves 97% accuracy. Additionally, with GAT, the other metric scores get closer to GNN-RE scores. The F1 score reaches 92%, precision 95%, and recall 90%, again validating our GAT implementation. We conducted these tests using the same configuration that was used for GNN-RE.

4.3 Configuration testing

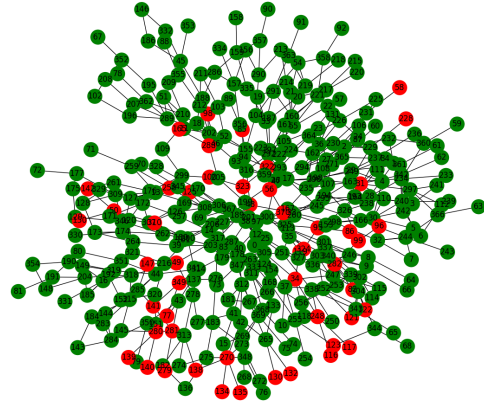
After the tests using GNN-RE data, the first step was to test the larger AISEC dataset with the same configuration. It quickly became evident that this configuration was not optimal for our data. Therefore, we ran a series of tests using different GNN



(a) Labeled graph after postprocessing with labels:
"sub": silver, "add": purple, "crc": light green,
"div": gold



(b) Label correctness before post processing



(c) Label correctness after post processing

Figure 4.2: Example subtractor graph from the reduced AISEC dataset with model predictions

and training parameter configurations, as we already addressed in section 3.3. The testing focused mainly on the hidden dimension parameter and the number of layers. Similar to what values were tested in GNN-RE [Alr+22], we tried three-, four-, and five-layer configurations and a hidden dimension of 64, 128, and 256. While these results were difficult to evaluate due to the variance in results for our dataset and only minor performance differences between different parameters, using a less complex model generally yielded better performance. This meant using a lower number of layers, specifically three layers, for our main tests. Aside from 64, which performed more poorly than the rest, there was very little difference in results for the hidden dimension values. For this reason, we used a hidden dimension of 256 for graphSAGE as configured in GNN-RE and 128 for GAT for more efficient testing as a compromise due to its high memory consumption. Similar results were observed for the training configuration, where the GAT model did not seem to keep improving after a certain number of epochs during tests, which resulted in our main tests for GAT training only for 500 epochs. During tests with graphSAGE, however, it could be observed that even during later epochs, very small improvements are made. For this reason, graphSAGE trains for the same 2000 epochs used in GNN-RE.

4.4 Full dataset tests

Our largest tests are run on the complete AISEC dataset, which includes 27 classes over 1,768 graphs. The results of these tests show that this data contains imbalances and overlaps. The highest accuracy that could be achieved with graphSAGE was 41% while other metrics stay as low as 20% as seen in table 4.1. Even the more advanced GAT model yields bad results. Its peak accuracy of 34% was lower than that of graphSAGE even though its other metrics scored a few percent higher in general. Independent of the used model and configuration, this dataset seems unfit for effective circuit classification in its current form.

4.5 reduced dataset tests

The expectations for tests in this dataset are higher since the subset was explicitly chosen to contain only 13 labels that seemed subjectively important, thereby avoiding the challenges of our dataset by removing many potential overlaps. As seen in table 4.1, the performance scores for this dataset were at least double compared to the full dataset. The peak accuracy of graphSAGE even went up to 80%. While other metrics still cannot be considered exceptionally good with a range of 56% to 71%, the application of the GNN is already way more usable than on the complete dataset. Inexplicably, the more

advanced GAT model with an attention mechanism performed around 10% worse in all metrics compared to graphSAGE in these tests. We rule out that this is due to the variance in the results since a considerable number of test runs were conducted. Further investigation into the reasons for this unexpected underperformance is out of this project’s scope.

4.6 arithmetic and core components dataset tests

The two smallest tested datasets showed various, partly unexpected, results. The arithmetic dataset contains only eight classes across 653 graphs, while the core components set comprises five classes across 594 graphs. When combined, these datasets form the reduced dataset, whose tests were described in section 4.5. Given this, a further reduction in data complexity is expected to enhance performance. However, this is not always true, as seen in table 4.1. Specifically, the graphSAGE model tests for the arithmetic subset show lower scores than those with the reduced set, achieving only an accuracy of 59%. Additionally, the graphSAGE tests on the core circuits set that should have the least data problems managed only a very slight increase in performance with an accuracy of 81% compared to the reduced set. On the other hand, scores for GAT tests have noticeably increased with these small datasets. While accuracy for the arithmetic set comes to a good 77% with GAT, the tests for the core components set, with an accuracy of 95%, show scores that come close to the exceptional results achieved with the GNN-RE data. These tests also satisfy the expectation that the more advanced GAT model should outperform the graphSAGE model.

4.7 osu035 dataset tests

The osu035 dataset is essentially the same as the reduced dataset whose tests were described in section 4.5. The notable difference is that only graphs synthesized using the osu035 cell library are used for training. Additionally, only graphs from the other three cell libraries, for example, osu018, are used for validation and testing. This reflects a realistic scenario where graphs that use a different technology than what was available as training data should be classified. While the hope was to see results comparable to the reduced dataset scores, the test performance, as seen in table 4.1, was very poor, with an accuracy of 35% for graphSAGE and 27% for GAT. As the results were even worse than with the full dataset, it seems that training on only one cell library does not work well with our GNN configuration. However, compared to the results of the similar reduced dataset, these results show a potential reason for the observed result

variance on different test runs since the random role assignment could amplify the problems caused by this technology disparity in the data.

5 Discussion

During our project, we gained a lot of perspective on graph neural networks and how to work with them.

There are some interesting discussion points arising from our results and our work on GNN.

As seen in section 4, our GNN achieves almost as good a result as [Alr+22], provided we use their dataset. However, using our own data sets provided by AISEC showed very different results. Here, only for subsets of our data that were very restricted, the results make the GNN implementation seem usable. Even in the best case on the smallest subset, test runs only achieved scores around 80% in metrics like f1-score, which is significantly lower than [Alr+22]’s scores of around 99%.

One apparent possible reason for this disparity in results is the different data quality. During our project, many challenges arose when using the large AISEC dataset. The unlabeled data, very general overlapping labels, and the varying availability of different circuit types made the data suboptimal for effective GNN training, even after much work toward reducing those problems. Compared to the results achieved with the GNN-RE data, this highlights the importance of fitting high-quality data. Unfortunately, such data is hardly available in a larger quantity. Therefore, while it seems our dataset was unfit for the used GNN concept, it should be considered that in a realistic larger-scale application scenario, available training data would look a lot more like our dataset than the seemingly required perfect data of GNN-RE, especially without an extreme amount of mostly manual preparation work on the data. This means while the general concept of GNN-RE is a very good solution to functional circuit reverse engineering, there is more work required to find a GNN implementation that can deal better with data that realistically is available in an actual application scenario and its challenges like more general labels and imbalances. Our tests using only training data from one cell library while trying to classify data from another, which is considered a realistic scenario, reinforce this conclusion, as those results were the worst across all our experiments.

Another approach that seems to improve results, as we could observe in our tests, is to specialize the scope of classification. Since our datasets using fewer labels achieved better performance, it could be argued that a useful approach for classifying realistic data is splitting the task into multiple different classifications, each trying to identify a smaller curated set of functionalities instead of finding one general solution for all

categories.

Aside from the significant impact of data challenges, one aspect of the GNN-RE concept we implemented proved very effective. Using random walk sampling for training with such a large dataset seems to work very well and even scales easily to vast data. For example, during our tests, it was possible to run training with a dataset of over 5.5 million nodes with a training time of only 1927s without deteriorating the overall GNN performance.

Therefore, our implementation proves that the overall approach presented by GNN-RE scales very well and can be helpful for vast data, even if further considerations are needed for using real nonsynthesized data.

6 Conclusion

Our IDP aimed to implement GNN-based circuit reverse engineering using the concepts presented in GNN-RE by Alrahis et al. (2022) [Alr+22]. While our implementation could successfully recreate the results from GNN-RE, another goal of this project was to test these concepts at a large scale with a dataset provided by Fraunhofer AISEC. Several challenges were encountered during the project when working with such vast data. Overcoming many of these problems has taught us a lot about the practical application of GNNs and the importance of fitting data. Our implementation could successfully show that extensive data can be scaled effectively by using random walk sampling. However, the mixed performance scores observed in our various test runs open up multiple new questions on how the GNN-RE concept can be applied to realistic data. The challenges encountered during the project and insights gained from the test results have revealed opportunities for future work, highlighting areas where further improvements can be made. Overall, we provided an implementation that can apply the GNN-RE concepts to large datasets and consider our IDP a success.

List of Figures

2.1	Dataset overview used by the authors, figure taken from [Alr+22]	7
2.2	GNN-RE node classification result, figure taken from [Alr+22]	7
3.1	Full feature vector for AISEC data	12
4.1	Metrics evolution during training of GAT on the reduced AISEC dataset	18
4.2	Example subtractor graph from the reduced AISEC dataset with model predictions	20

List of Tables

2.1	GNN-RE configuration details, table taken from [Alr+22]	6
3.1	GNN-RE dataset [Alr+22] differences from feature implementation . . .	10
3.2	AISEC dataset and subsets	11
3.3	GNN architecture and training configuration details	15
4.1	GNN performance results of AISEC dataset and subsets	17

Bibliography

- [Alr+22] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu. "GNN-RE: Graph Neural Networks for Reverse Engineering of Gate-Level Netlists." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41.8 (2022), pp. 2435–2448. doi: 10.1109/TCAD.2021.3110807.
- [Bot+21] U. J. Botero, R. Wilson, H. Lu, M. T. Rahman, M. A. Mallaiyan, F. Ganji, N. Asadizanjani, M. M. Tehranipoor, D. L. Woodard, and D. Forte. "Hardware Trust and Assurance through Reverse Engineering: A Tutorial and Outlook from Image Analysis and Machine Learning Perspectives." In: *J. Emerg. Technol. Comput. Syst.* 17.4 (June 2021). issn: 1550-4832. doi: 10.1145/3464959.
- [DB17] Y.-Y. Dai and R. K. Brayton. "Circuit recognition with deep learning." In: *2017 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 2017, pp. 162–162. doi: 10.1109/HST.2017.7951826.
- [Fay+19] A. Fayyazi, S. Shababi, P. Nuzzo, S. Nazarian, and M. Pedram. "Deep Learning-Based Circuit Recognition Using Sparse Mapping and Level-Dependent Decaying Sum Circuit Representations." In: *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2019, pp. 638–641. doi: 10.23919/DATE.2019.8715251.
- [Hon+21] X. Hong, T. Lin, Y. Shi, and B. H. Gwee. "ASIC Circuit Netlist Recognition Using Graph Neural Network." In: *2021 IEEE International Symposium on the Physical and Failure Analysis of Integrated Circuits (IPFA)*. 2021, pp. 1–5. doi: 10.1109/IPFA53173.2021.9617311.
- [HYL17] W. Hamilton, Z. Ying, and J. Leskovec. "Inductive representation learning on large graphs." In: *Advances in neural information processing systems* 30 (2017).
- [Kun+20] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar. "GANA: Graph Convolutional Network Based Automated Netlist Annotation for Analog Circuits."

- In: *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2020, pp. 55–60. doi: 10.23919/DATE48585.2020.9116329.
- [Li+20] Y. Li, Y. Lin, M. Madhusudan, A. Sharma, W. Xu, S. S. Sapatnekar, R. Harjani, and J. Hu. “A customized graph neural network model for guiding analog IC placement.” In: *Proceedings of the 39th International Conference on Computer-Aided Design*. ICCAD ’20. Virtual Event, USA: Association for Computing Machinery, 2020. isbn: 9781450380263. doi: 10.1145/3400302.3415624.
- [McL11] I. McLoughlin. “Reverse engineering of embedded consumer electronic systems.” In: *2011 IEEE 15th International Symposium on Consumer Electronics (ISCE)*. 2011, pp. 352–356. doi: 10.1109/ISCE.2011.5973848.
- [RSK14] J. Rajendran, O. Sinanoglu, and R. Karri. “Regaining Trust in VLSI Design: Design-for-Trust Techniques.” In: *Proceedings of the IEEE* 102.8 (2014), pp. 1266–1282. doi: 10.1109/JPROC.2014.2332154.
- [Sil+18] L. M. Silva, F. V. Andrade, A. O. Fernandes, and L. F. M. Vieira. “Arithmetic Circuit Classification Using Convolutional Neural Networks.” In: *2018 International Joint Conference on Neural Networks (IJCNN)*. 2018, pp. 1–7. doi: 10.1109/IJCNN.2018.8489382.
- [Vel+17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. “Graph attention networks.” In: *arXiv preprint arXiv:1710.10903* (2017).
- [Zen+19] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna. “Graphsaint: Graph sampling based inductive learning method.” In: *arXiv preprint arXiv:1907.04931* (2019).